

# AGNEL INSTITUTE OF TECHNOLOGY AND DESIGN

---

## AND Gate:

```
import numpy as np

# Bipolar AND gate inputs and targets
inputs = np.array([
    [1, 1, 1], # x1, x2, bias
    [1, -1, 1],
    [-1, 1, 1],
    [-1, -1, 1]
])

targets = np.array([1, -1, -1, -1]) # Bipolar targets

def activation(yin):
    if yin > 0:
        return 1
    elif yin == 0:
        return 0
    else:
        return -1

def get_user_input():
    print("Enter initial weights and bias:")
    while True:
        try:
            w1 = float(input("Initial weight w1: "))
            w2 = float(input("Initial weight w2: "))
            b = float(input("Initial bias weight: "))
            alpha = float(input("Learning rate ( $\alpha$ ): "))
            break
```

# AGNEL INSTITUTE OF TECHNOLOGY AND DESIGN

---

```
except ValueError:
```

```
    print("Invalid input. Please enter numeric values.")
```

```
return np.array([w1, w2, b], dtype=float), alpha
```

```
def train_perceptron_bipolar(inputs, targets, alpha, weights, epochs=2):
```

```
    for epoch in range(epochs):
```

```
        print(f"\nEpoch {epoch + 1}")
```

```
        print("-" * 90)
```

```
        print("x1 x2 b | T | yin | y |  $\Delta w_1$   $\Delta w_2$   $\Delta b$  | w1(new) w2(new) b(new)")
```

```
        print("-" * 90)
```

```
    for i in range(len(inputs)):
```

```
        x = inputs[i]
```

```
        t = targets[i]
```

```
        yin = np.dot(x, weights)
```

```
        y = activation(yin)
```

```
        if y != t:
```

```
            delta_w = alpha * t * x
```

```
            weights += delta_w
```

```
        else:
```

```
            delta_w = [0, 0, 0]
```

```
        print(f"{x[0]:>2} {x[1]:>2} {x[2]:>2} | {t:>2} | {yin:>3} | {y:>2} | "
```

```
              f" {delta_w[0]:>3} {delta_w[1]:>3} {delta_w[2]:>3} | "
```

```
              f" {weights[0]:>3} {weights[1]:>3} {weights[2]:>3}")
```

```
    return weights
```

```
# Get user input
```

# AGNEL INSTITUTE OF TECHNOLOGY AND DESIGN

```
weights, alpha = get_user_input()
```

```
# Train for 2 epochs
```

```
final_weights = train_perceptron_bipolar(inputs, targets, alpha, weights, epochs=2)
```

```
# Final results
```

```
print("\nFinal weights after training:")
```

```
print(f"w1 = {final_weights[0]}, w2 = {final_weights[1]}, bias_weight = {final_weights[2]}")
```

## Output:

```
Enter initial weights and bias:
```

```
Initial weight w1: 0
```

```
Initial weight w2: 0
```

```
Initial bias weight: 0
```

```
Learning rate ( $\alpha$ ): 1
```

```
Epoch 1
```

| x1 | x2 | b | T  | yin  | y  | $\Delta w1$ | $\Delta w2$ | $\Delta b$ | w1(new) | w2(new) | b(new) |
|----|----|---|----|------|----|-------------|-------------|------------|---------|---------|--------|
| 1  | 1  | 1 | 1  | 0.0  | 0  | 1.0         | 1.0         | 1.0        | 1.0     | 1.0     | 1.0    |
| 1  | -1 | 1 | -1 | 1.0  | 1  | -1.0        | 1.0         | -1.0       | 0.0     | 2.0     | 0.0    |
| -1 | 1  | 1 | -1 | 2.0  | 1  | 1.0         | -1.0        | -1.0       | 1.0     | 1.0     | -1.0   |
| -1 | -1 | 1 | -1 | -3.0 | -1 | 0           | 0           | 0          | 1.0     | 1.0     | -1.0   |

```
Epoch 2
```

| x1 | x2 | b | T  | yin  | y  | $\Delta w1$ | $\Delta w2$ | $\Delta b$ | w1(new) | w2(new) | b(new) |
|----|----|---|----|------|----|-------------|-------------|------------|---------|---------|--------|
| 1  | 1  | 1 | 1  | 1.0  | 1  | 0           | 0           | 0          | 1.0     | 1.0     | -1.0   |
| 1  | -1 | 1 | -1 | -1.0 | -1 | 0           | 0           | 0          | 1.0     | 1.0     | -1.0   |
| -1 | 1  | 1 | -1 | -1.0 | -1 | 0           | 0           | 0          | 1.0     | 1.0     | -1.0   |
| -1 | -1 | 1 | -1 | -3.0 | -1 | 0           | 0           | 0          | 1.0     | 1.0     | -1.0   |

```
Final weights after training:
```

```
w1 = 1.0, w2 = 1.0, bias_weight = -1.0
```

# AGNEL INSTITUTE OF TECHNOLOGY AND DESIGN

---

## OR Gate:

```
import numpy as np

# Bipolar OR gate inputs and targets
inputs = np.array([
    [1, 1, 1], # x1, x2, bias
    [1, -1, 1],
    [-1, 1, 1],
    [-1, -1, 1]
])

targets = np.array([1, 1, 1, -1]) # Bipolar targets for OR gate

def activation(yin):
    if yin > 0:
        return 1
    elif yin == 0:
        return 0
    else:
        return -1

def get_user_input():
    print("Enter initial weights and bias:")
    while True:
        try:
            w1 = float(input("Initial weight w1: "))
            w2 = float(input("Initial weight w2: "))
            b = float(input("Initial bias weight: "))
            alpha = float(input("Learning rate ( $\alpha$ ): "))
            break
```

# AGNEL INSTITUTE OF TECHNOLOGY AND DESIGN

---

```
except ValueError:
```

```
    print("Invalid input. Please enter numeric values.")
```

```
return np.array([w1, w2, b], dtype=float), alpha
```

```
def train_perceptron_bipolar(inputs, targets, alpha, weights, epochs=2):
```

```
    for epoch in range(epochs):
```

```
        print(f"\nEpoch {epoch + 1}")
```

```
        print("-" * 90)
```

```
        print("x1 x2 b | T | yin | y |  $\Delta w_1$   $\Delta w_2$   $\Delta b$  | w1(new) w2(new) b(new)")
```

```
        print("-" * 90)
```

```
    for i in range(len(inputs)):
```

```
        x = inputs[i]
```

```
        t = targets[i]
```

```
        yin = np.dot(x, weights)
```

```
        y = activation(yin)
```

```
        if y != t:
```

```
            delta_w = alpha * t * x
```

```
            weights += delta_w
```

```
        else:
```

```
            delta_w = [0, 0, 0]
```

```
        print(f"{x[0]:>2} {x[1]:>2} {x[2]:>2} | {t:>2} | {yin:>3} | {y:>2} | "
```

```
              f" {delta_w[0]:>3} {delta_w[1]:>3} {delta_w[2]:>3} | "
```

```
              f" {weights[0]:>3} {weights[1]:>3} {weights[2]:>3}")
```

```
    return weights
```

```
# Get user input
```

# AGNEL INSTITUTE OF TECHNOLOGY AND DESIGN

```
weights, alpha = get_user_input()
```

```
# Train for 2 epochs
```

```
final_weights = train_perceptron_bipolar(inputs, targets, alpha, weights, epochs=2)
```

```
# Final results
```

```
print("\nFinal weights after training:")
```

```
print(f"w1 = {final_weights[0]}, w2 = {final_weights[1]}, bias_weight = {final_weights[2]}")
```

## Output:

```
Enter initial weights and bias:
```

```
Initial weight w1: 0
```

```
Initial weight w2: 0
```

```
Initial bias weight: 0
```

```
Learning rate ( $\alpha$ ): 1
```

```
Epoch 1
```

```
x1 x2 b | T | yin | y |  $\Delta w1$   $\Delta w2$   $\Delta b$  | w1(new) w2(new) b(new)
```

|    |    |   |  |    |  |      |  |    |  |     |     |     |  |     |     |     |
|----|----|---|--|----|--|------|--|----|--|-----|-----|-----|--|-----|-----|-----|
| 1  | 1  | 1 |  | 1  |  | 0.0  |  | 0  |  | 1.0 | 1.0 | 1.0 |  | 1.0 | 1.0 | 1.0 |
| 1  | -1 | 1 |  | 1  |  | 1.0  |  | 1  |  | 0   | 0   | 0   |  | 1.0 | 1.0 | 1.0 |
| -1 | 1  | 1 |  | 1  |  | 1.0  |  | 1  |  | 0   | 0   | 0   |  | 1.0 | 1.0 | 1.0 |
| -1 | -1 | 1 |  | -1 |  | -1.0 |  | -1 |  | 0   | 0   | 0   |  | 1.0 | 1.0 | 1.0 |

```
Epoch 2
```

```
x1 x2 b | T | yin | y |  $\Delta w1$   $\Delta w2$   $\Delta b$  | w1(new) w2(new) b(new)
```

|    |    |   |  |    |  |      |  |    |  |   |   |   |  |     |     |     |
|----|----|---|--|----|--|------|--|----|--|---|---|---|--|-----|-----|-----|
| 1  | 1  | 1 |  | 1  |  | 3.0  |  | 1  |  | 0 | 0 | 0 |  | 1.0 | 1.0 | 1.0 |
| 1  | -1 | 1 |  | 1  |  | 1.0  |  | 1  |  | 0 | 0 | 0 |  | 1.0 | 1.0 | 1.0 |
| -1 | 1  | 1 |  | 1  |  | 1.0  |  | 1  |  | 0 | 0 | 0 |  | 1.0 | 1.0 | 1.0 |
| -1 | -1 | 1 |  | -1 |  | -1.0 |  | -1 |  | 0 | 0 | 0 |  | 1.0 | 1.0 | 1.0 |

```
Final weights after training:
```

```
w1 = 1.0, w2 = 1.0, bias_weight = 1.0
```